

CO4 ASSESSMENT TOOL 2 – CI/CD PIPELINE DESIGN EXERCISE

Fraud Detection and Financial Transaction Monitoring System

Field	Details	Field	Details
Course Code	CSA10 / CSA1063	Course Title	Software Engineering
CO Assessed	CO4	Assessment	CI/CD Pipeline Design Exercise
Weightage	20%	Total Marks	25
Student Name	Jeyakannan L	Reg. No.	192525376
Date	24/08/2026	Domain	Financial Technology / Fraud Monitoring

Code of Conduct

I Jeyakannan L (192525376) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Signature of the Student: Jeyakannan L

Rubrics for Calculation

Criteria	Mark
Requirement & Pipeline Analysis (15%)	/5
Pipeline Architecture & Workflow (25%)	/4
Tools & Technology Selection (20%)	/4
Automated Testing & Quality Integration (15%)	/4
Deployment, Security & Rollback Strategy (15%)	/4
Pipeline Diagram & Documentation (10%)	/4
TOTAL	/25

Annexure A – CI/CD Pipeline Design Exercise

1. Analyze the Project Requirements

1.1 Project Overview

The Fraud Detection and Financial Transaction Monitoring System is a web-based platform that monitors financial transactions, evaluates suspicious activity and presents fraud-risk alerts to authorized users.

- Frontend: React + Vite – monitoring dashboard and alert views.
- Backend: Python Flask – REST APIs, validation and fraud-analysis services.
- Database: PostgreSQL – transaction, user, risk and alert records.
- Fraud Analysis: Python rules/scoring module for suspicious transactions.
- Containerization: Docker.
- Version Control: Git + GitHub.
- CI/CD: GitHub Actions.
- Deployment: Docker Compose.

1.2 Why CI/CD is Required

CI/CD is required because changes to a fraud-monitoring system must be checked for correctness, security and reliability before release. Automation reduces manual mistakes and provides a repeatable path from source-code change to deployment.

Without CI/CD: Developer → Write Code → Manually Test → Build → Deploy

With CI/CD: Developer → Git Push → Build → Automated Tests → Quality Check → Security Scan → Docker Build → Staging → Smoke Test → Production → Monitoring

Fraud Monitoring CI/CD Pipeline

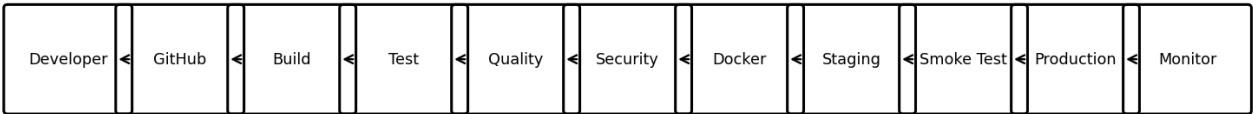


Figure 1: Proposed CI/CD pipeline for the fraud-monitoring system.

2. CI/CD Pipeline Workflow

Stage	Purpose	Gate
Source	Retrieve approved source code.	Checkout successful
Build	Install dependencies and build application.	Build successful
Automated Testing	Run unit, API and UI tests.	Tests pass
Quality Analysis	Lint and static checks.	No critical quality issue
Security Scan	Scan dependencies, secrets and Docker images.	No critical vulnerability

Docker Build	Create versioned images.	Images build
Staging	Deploy candidate release.	Services healthy
Smoke Testing	Verify critical user/API/fraud flows.	Smoke tests pass
Production	Promote validated release.	Approval
Monitoring	Observe health and errors.	Stable
Rollback	Restore previous version if required.	Previous image available

3. Tools and Technologies

Stage	Tool	Reason
Source Control	Git	Version history and branches
Repository	GitHub	Collaboration and pull requests
CI/CD	GitHub Actions	Integrated workflow automation
Frontend	React + Vite	Modern web UI/build
Backend	Python + Flask	REST API and fraud-analysis integration
Testing	pytest + Vitest	Automated backend/frontend tests
Quality	ESLint	Static code-quality checks
Database	PostgreSQL	Structured transaction storage
Containerization	Docker	Consistent environments
Deployment	Docker Compose	Multi-container deployment
Security	Dependabot + Trivy	Dependency/image vulnerability checks
Monitoring	Health checks + logs	Post-deployment visibility

4. Automated Testing Strategy

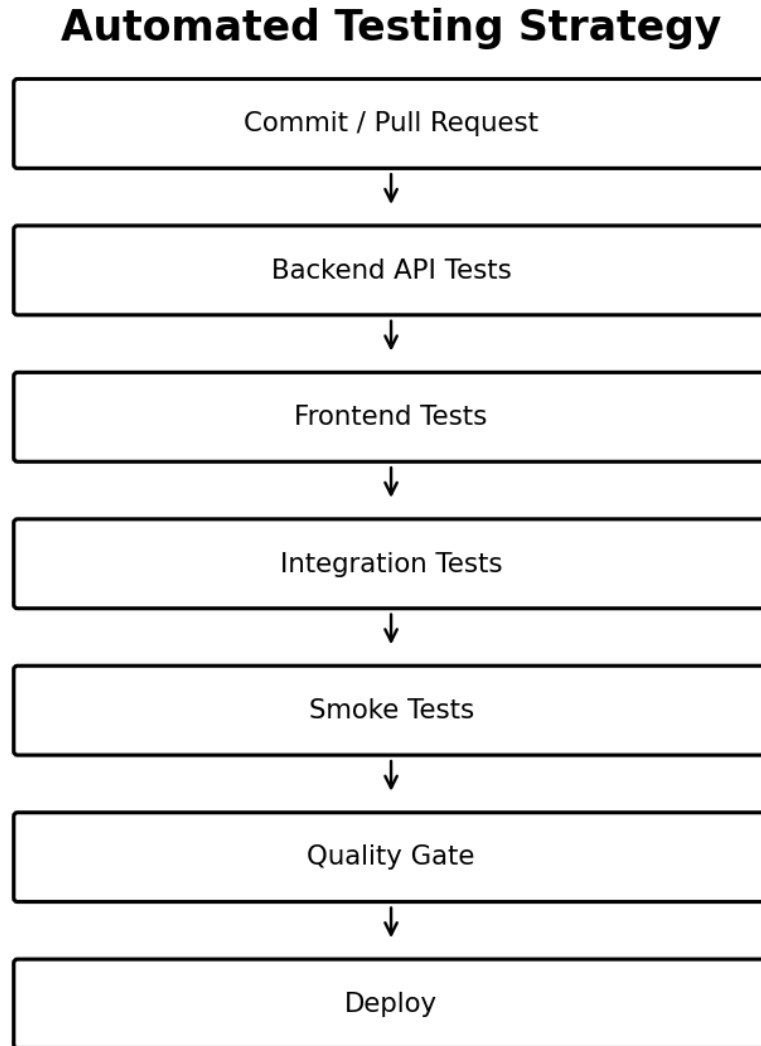


Figure 2: Automated testing and quality-gate strategy.

4.1 Backend Testing

- Application health endpoint responds successfully.
- Authentication API accepts valid credentials and rejects invalid credentials.
- Transaction API returns valid JSON and correct status codes.
- Transaction validation rejects missing, negative or malformed values.
- Fraud-analysis service returns a valid risk score/risk level.
- High-risk transactions generate a linked alert.
- Invalid API requests return suitable error responses.

4.2 Frontend Testing

- Login page renders correctly.
- Transaction dashboard loads data.
- Search/filter controls work.
- Risk score and alert information are displayed.

- Invalid form inputs show validation feedback.

4.3 Integration Testing

React Frontend → Flask API → Fraud Analysis / PostgreSQL → JSON Response → Dashboard

Integration tests verify that transaction data travels correctly between frontend, backend, analysis logic and database.

4.4 Smoke Testing

- Open application.
- Check dashboard/login loads.
- Check backend health.
- Load a transaction.
- Run fraud analysis.
- Verify a high-risk test transaction creates an alert.

A critical smoke-test failure should stop production promotion or trigger rollback.

5. Deployment Strategy

Environment	Purpose	Promotion
Development	Feature implementation and developer testing.	Feature branch → development
Staging	Final integration/security/smoke testing.	Successful CI build only
Production	Live monitoring environment.	Validated version + approval

Environment Promotion Flow



Figure 3: Environment promotion flow.

Flow: Feature Branch → Development → Testing → Staging → Validation → Approval → Production → Monitoring

6. Security, Quality and Rollback

6.1 Security

- Dependency vulnerability scanning.
- Docker image scanning.
- Secrets stored in GitHub Actions Secrets or a secure manager.
- No passwords/API keys committed to GitHub.
- Environment variables for sensitive configuration.
- HTTPS/TLS in production.

- Authentication and role-based access control.
- Audit logs for important fraud-alert actions.
- Least-privilege CI/CD permissions.

6.2 Quality Checks

Checkout → Linting → Unit Tests → Integration Tests → Security Scan → Docker Build → Staging → Smoke Tests

If a critical check fails: Pipeline FAILED → Deployment STOPPED.

6.3 Rollback Mechanism

Every production release uses a versioned Docker image. If the new release fails, the previous stable image is restored.

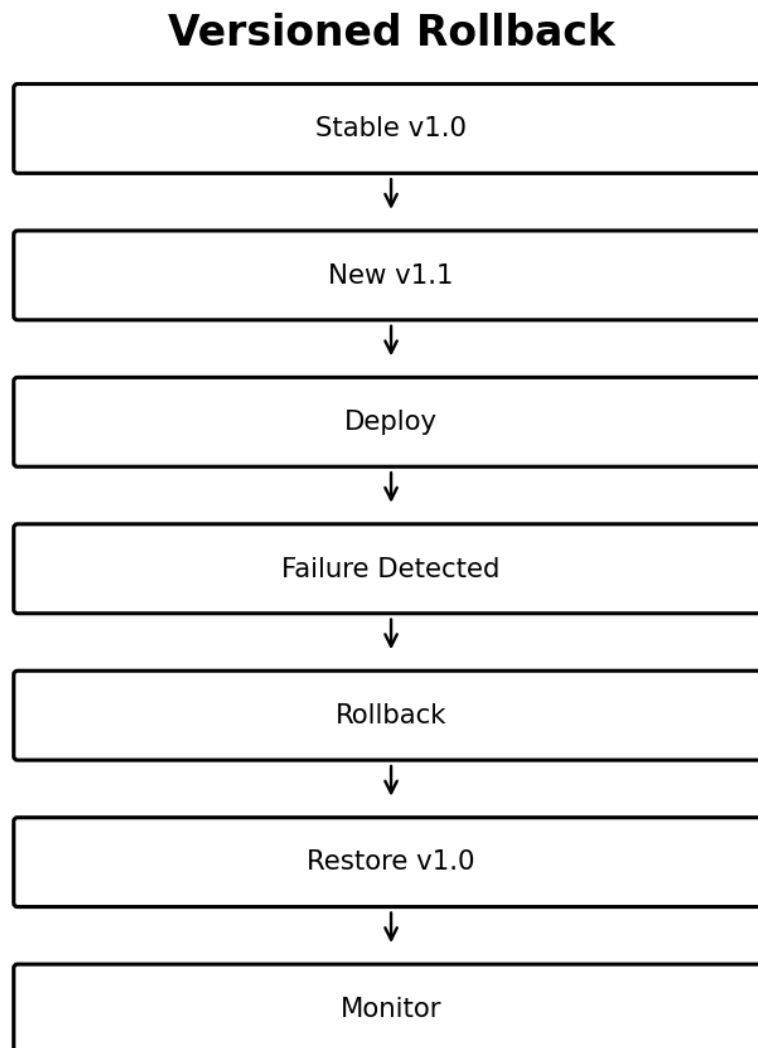


Figure 4: Versioned rollback mechanism.

7. CI/CD Pipeline Architecture

Fraud Monitoring CI/CD Pipeline

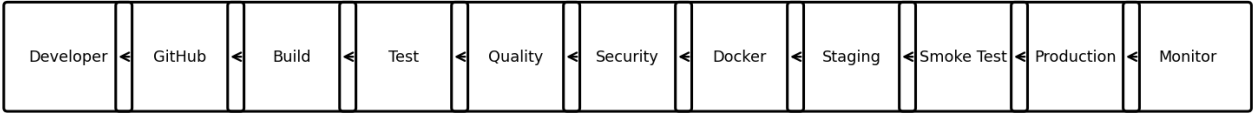


Figure 5: Complete CI/CD architecture and release flow.

Component	Responsibility
Developer	Creates and reviews source changes.
GitHub	Repository, branches and pull requests.
GitHub Actions	Runs CI/CD jobs.
Test Layer	Backend, frontend and integration checks.
Quality Layer	Linting/static checks.
Security Layer	Dependency, secret and image scanning.
Docker	Packages services into images.
Staging	Validates release before production.
Production	Runs approved version.
Monitoring	Detects health and error conditions.

8. Example GitHub Actions Workflow

```
name: Fraud Monitoring CI/CD

on:
  push:
    branches: [main, development]
  pull_request:
    branches: [main, development]

jobs:
  backend-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r backend/requirements.txt
      - run: pytest backend/tests

  frontend-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: "22"
      - run: npm install
        working-directory: frontend
      - run: npm run build
        working-directory: frontend
```

CO4 AT2 – CI/CD Pipeline Design | Fraud Detection and Financial Transaction Monitoring System | Jeyakannan L – 192525376

```

docker-build:
  needs: [backend-test, frontend-build]
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: docker compose build

```

9. Workflow Explanation

Step 1 – Code Commit

Developer pushes a branch or pull request to GitHub.

Step 2 – Checkout

GitHub Actions retrieves the latest source.

Step 3 – Build

Frontend and backend dependencies are installed and built.

Step 4 – Testing

Backend, frontend and integration tests run automatically.

Step 5 – Quality Analysis

Linting/static checks identify code-quality problems.

Step 6 – Security

Dependencies and Docker images are scanned.

Step 7 – Docker Packaging

Application services are converted into versioned images.

Step 8 – Staging

Candidate release is deployed to staging.

Step 9 – Smoke Testing

Critical transaction and fraud-alert paths are verified.

Step 10 – Production

Only a validated version is promoted.

Step 11 – Monitoring and Rollback

Health/log monitoring continues; critical failure restores the previous version.

10. Justification

The proposed CI/CD approach is suitable because GitHub Actions integrates naturally with GitHub, React/Vite and Flask can be independently built and tested, PostgreSQL provides persistent transaction storage, and Docker/Docker Compose provide repeatable environments. Automated testing improves reliability, security scanning reduces deployment risk, and versioned images provide rollback capability.

Expected Improvement	How CI/CD Helps
Development speed	Automatic build/test reduces repetitive manual work.

Code quality	Linting and automated tests detect defects early.
Reliability	Integration and smoke tests prevent clearly broken releases.
Security	Dependency and image scanning reduces risk.
Deployment consistency	Docker provides repeatable environments.
Maintainability	Structured workflow stages are easy to understand.
Rollback capability	Previous version can be restored quickly.
Collaboration	GitHub branches and pull requests support review.

11. Evidence / Screenshot Placement Guide

- Figure 6 – GitHub Actions workflow run showing build and test jobs.
- Figure 7 – Automated backend/frontend test results.
- Figure 8 – Docker containers running successfully.
- Figure 9 – Staging application and smoke-test result.
- Figure 10 – Security scan result.
- Figure 11 – Successful deployment status.
- Figure 12 – Rollback demonstration, if performed.

The diagrams included above are design diagrams. For the final submission, add real screenshots from your own project execution where evidence is required.

12. Rubric Alignment Checklist

Rubric Criterion	Evidence Included	Target
Requirement & Pipeline Analysis (15%)	Project overview, CI/CD need, components and requirements.	Excellent
Pipeline Architecture & Workflow (25%)	Complete staged workflow, architecture and environment diagrams.	Excellent
Tools & Technology Selection (20%)	Technology table with purpose and justification.	Excellent
Automated Testing & Quality Integration (15%)	Backend, frontend, integration, smoke tests and gates.	Excellent
Deployment, Security & Rollback Strategy (15%)	Three environments, security controls and versioned rollback.	Excellent
Pipeline Diagram & Documentation (10%)	Multiple diagrams, tables and workflow explanations.	Excellent